YUVAINTERN – PYTHON DEVELOPER INTERNSHIP

TASK 2: CORE PYTHON CONCEPTS AND PRACTICAL PROJECTS

Student Name: [T.Nandhini]

Course: B.Tech – Computer Science and Engineering (AI & ML)

Internship Role: Python Developer Intern

Internship Duration: 8 Weeks

Organization: YuvaIntern

---

1. INTRODUCTION

Python is a high-level, interpreted, general-purpose programming language known for its simple syntax and readability. It is widely used in software development, data science, artificial intelligence, machine learning, and web development.

This task focuses on learning Python fundamentals and applying programming concepts through practical exercises and mini-projects.

2. OBJECTIVES

- Understand Python fundamentals and syntax.
- Learn variables and different data types.
- Perform operations using Python operators.
- Implement conditional statements and loops.
- Create and use functions.
- Understand Python data structures.
- Learn object-oriented programming.
- Handle exceptions effectively.
- Perform file handling operations.
- Develop practical Python applications.

3. SOFTWARE AND TOOLS USED

Tool	Purpose
Python 3.14	Programming language
Visual Studio Code	Code editor
Git	Version control
GitHub	Repository management
Windows 11	Operating system

4. PYTHON FUNDAMENTALS

This section covers the basic concepts of Python programming.

Topics covered:

- Variables
- Data types
- Strings
- Numbers
- Type conversion
- Variable scope
- User-defined variables and values

Practical exercises were performed to understand how Python stores and processes information.

5. OPERATORS

Python operators are used to perform operations on values and variables.

Types covered:

- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- Membership operators

These operators were implemented through practical programs.

6. CONDITIONAL STATEMENTS

Conditional statements allow programs to make decisions based on conditions.

Topics covered:

- if statement
- if-else statement
- if-elif-else statement
- Nested if statements

Programs were developed to understand decision-making in Python.

7. LOOPS

Loops are used to execute a block of code repeatedly.

Topics covered:

- for loop
- while loop
- break statement
- continue statement
- Nested loops

Practical programs were executed to understand repetition and iteration.

8. FUNCTIONS

Functions are reusable blocks of code that perform specific tasks.

Topics covered:

- Function definition
- Function calling
- Parameters and arguments
- Return statements
- Default arguments
- Keyword arguments
- Recursive functions

9. DATA STRUCTURES

Python provides different data structures to store and organize information.

The following were practiced:

- Lists
- Tuples
- Sets
- Dictionaries

Operations such as adding, removing, accessing, and updating elements were implemented.

10. OBJECT-ORIENTED PROGRAMMING

Object-oriented programming organizes code using classes and objects.

Concepts covered:

- Classes and objects
- Constructors
- Inheritance
- Polymorphism
- Encapsulation

Practical programs were developed to understand object-oriented programming concepts.

11. EXCEPTION HANDLING

Exception handling helps manage runtime errors and prevents unexpected program termination.

Topics covered:

- try and except blocks
- Handling multiple exceptions
- finally block
- raise statement

Different examples were executed to understand error handling.

12. FILE HANDLING

File handling allows Python programs to create, read, write, and update files.

Operations performed:

- Creating files
- Writing data into files
- Reading file contents
- Appending data to files

These operations were practiced using Python file handling methods.

13. PRACTICAL PROJECTS

Three practical projects were developed as part of this task.

13.1 Student Marks Management System

This project manages student marks and calculates academic results using Python programming concepts.

13.2 Contact Management System

This project helps manage contact information, including adding and displaying contact details.

13.3 Library Management System

This project demonstrates basic library operations, including managing book information and records.

14. PRACTICE PROGRAMS

The following practice programs were implemented:

1. Even or Odd Number
2. Factorial
3. Prime Number
4. Palindrome
5. Fibonacci Series

These exercises improved logical thinking and programming skills.

15. TESTING AND SCREENSHOTS

All developed Python programs were executed using Visual Studio Code.

Screenshots of program execution were captured and organized in the project folder.

Test cases were prepared to verify the expected behavior of the programs.

PROGRAM EXECUTION SCREENSHOTS

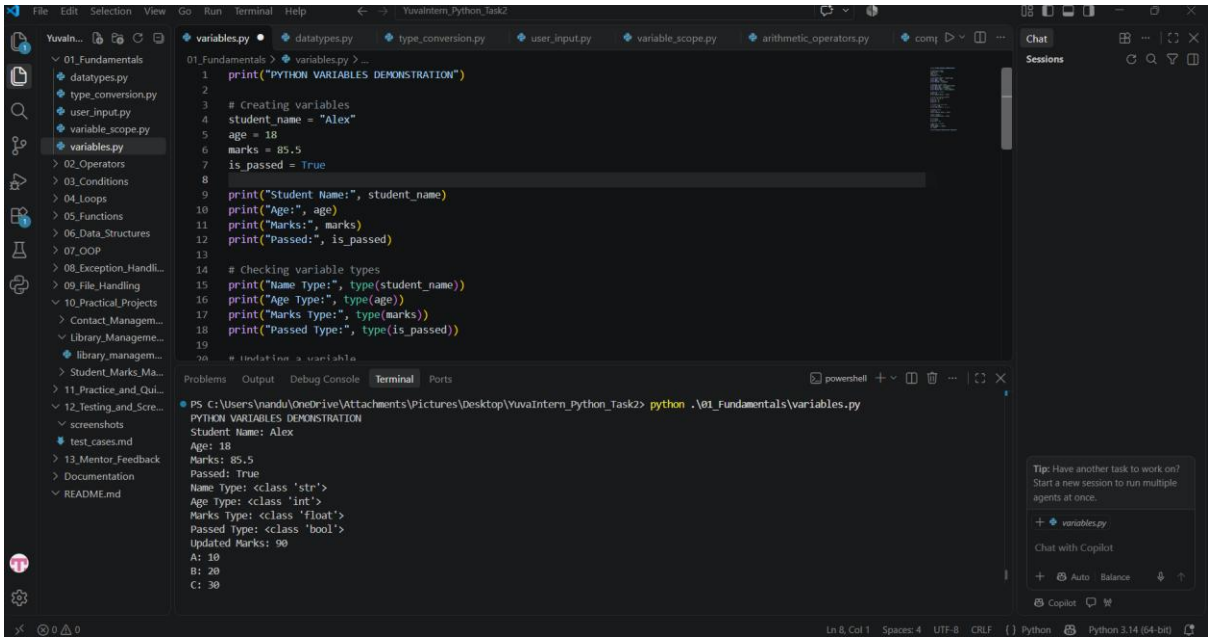


Figure 1: Execution of Python Variables Program

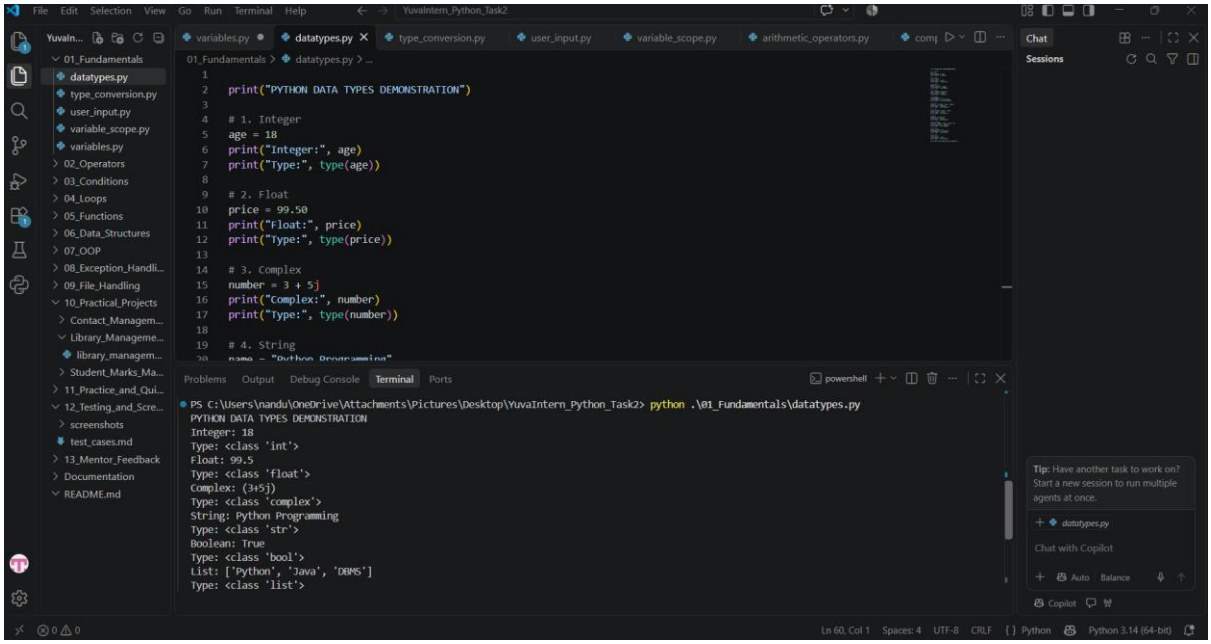


Figure 2: Execution of Python Data Types Program

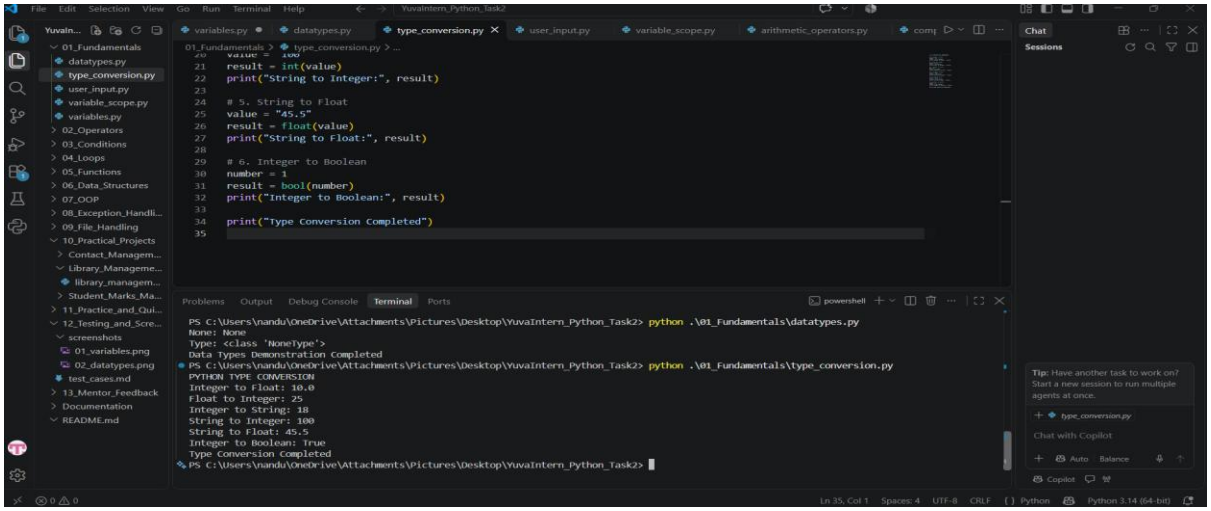


Figure 3: Type Conversion

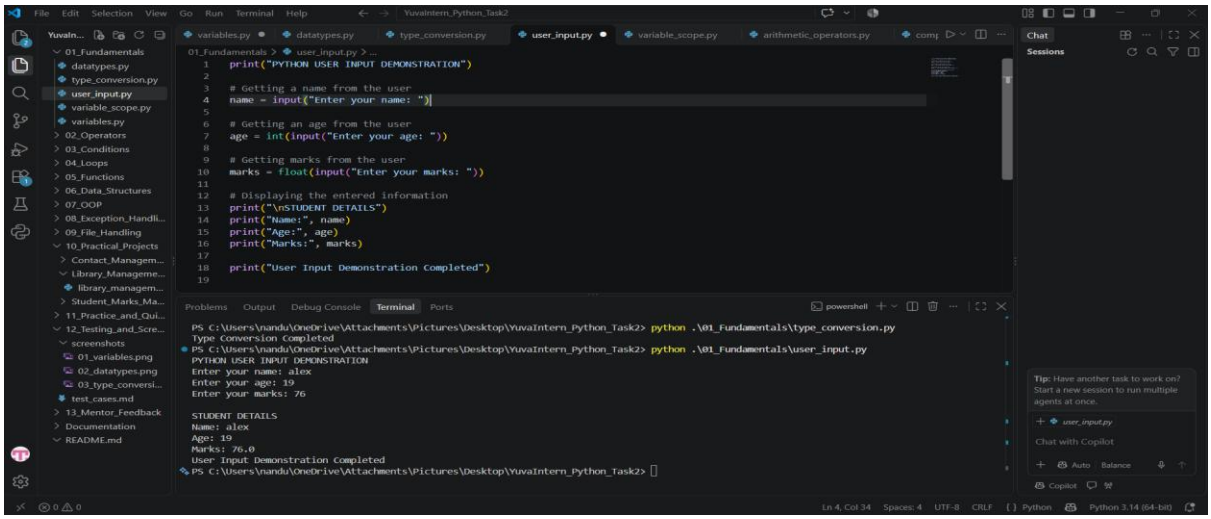


Figure 4: Arithmetic Operators

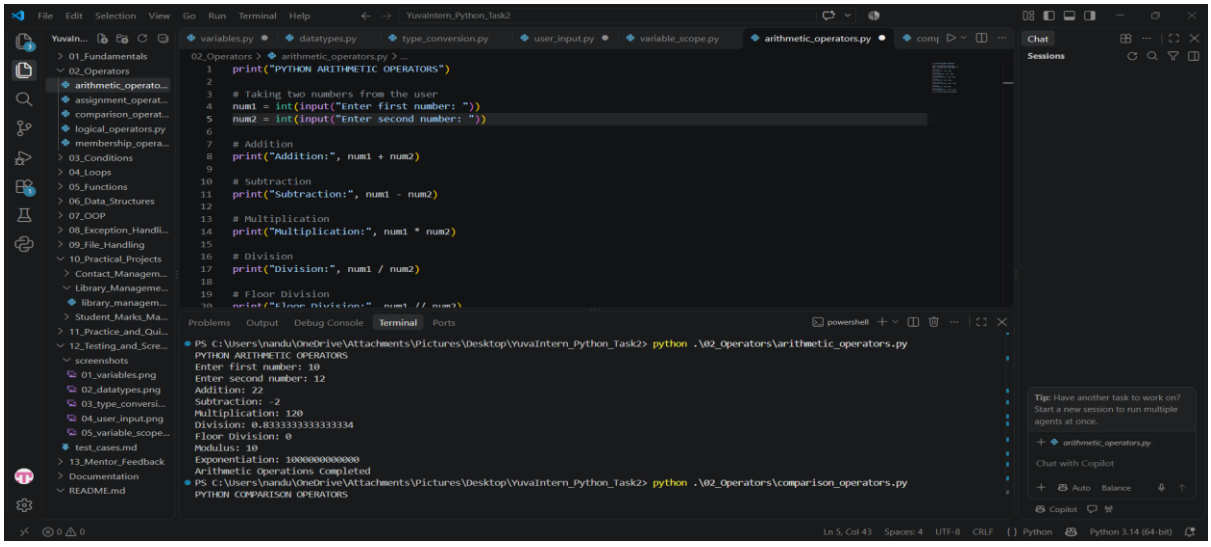


Figure 5: For Loop

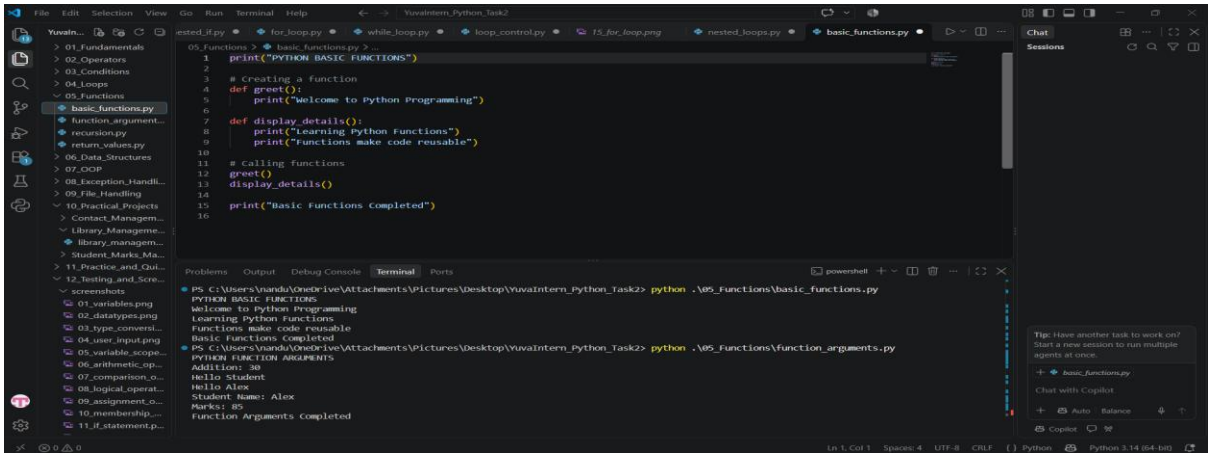


Figure 6: Functions

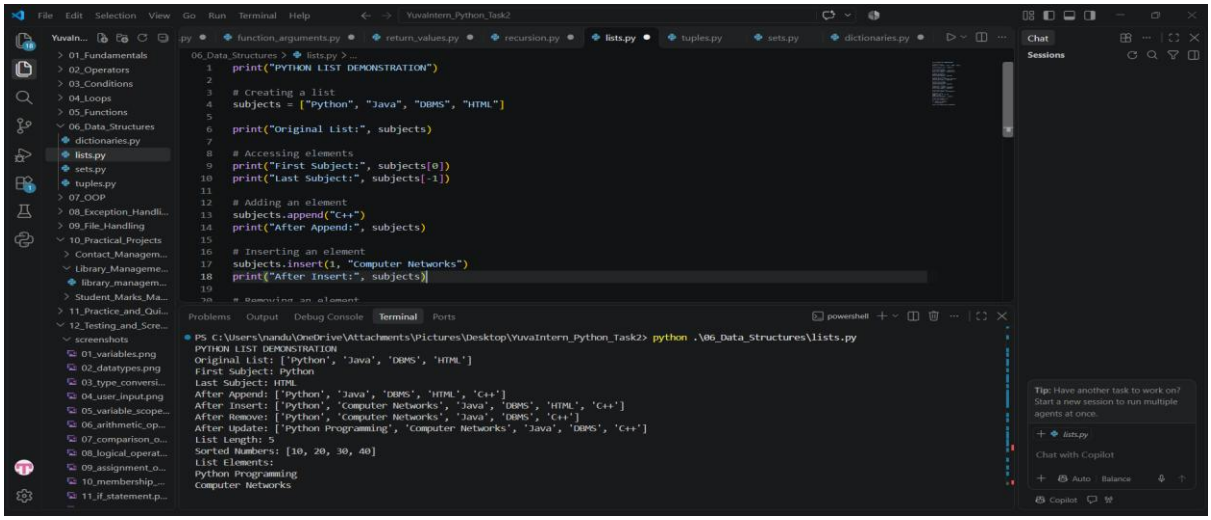


Figure 7: Lists

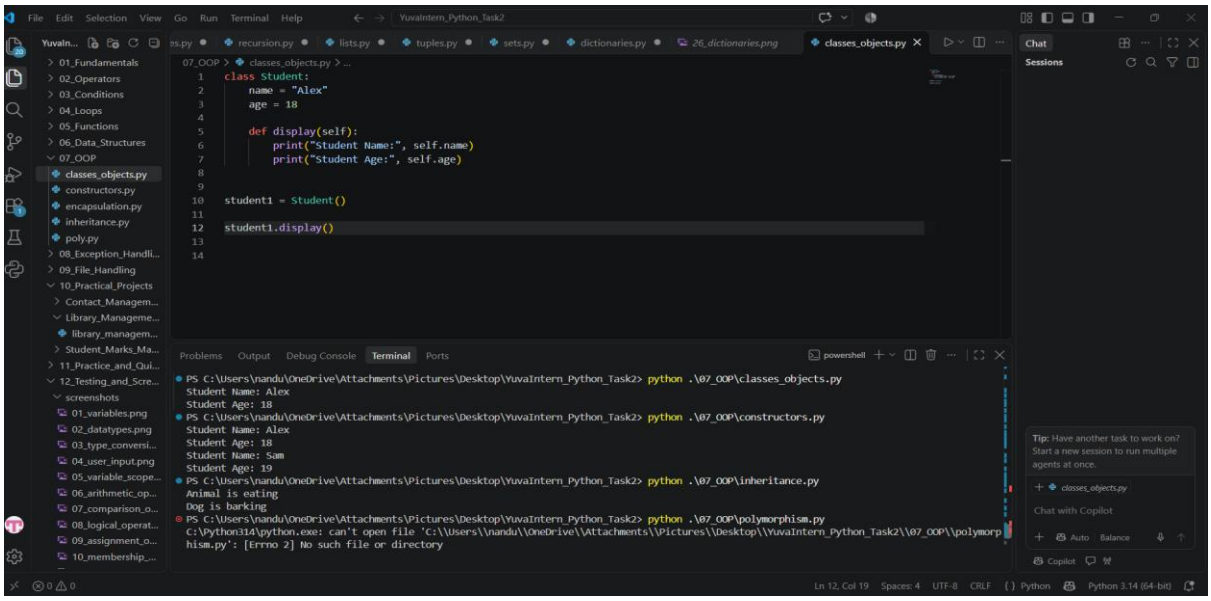


Figure 8: Classes and Objects

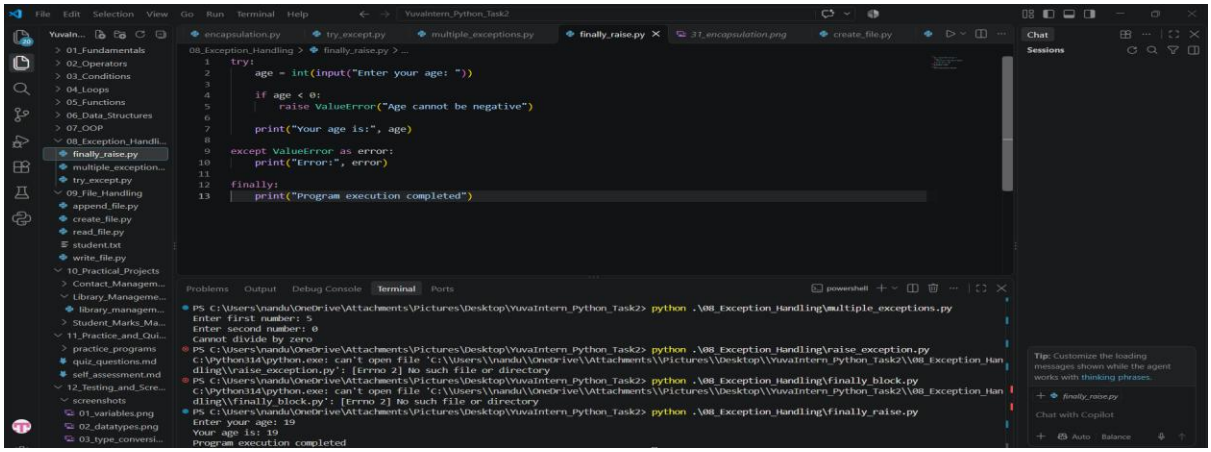


Figure 9: Exception Handling

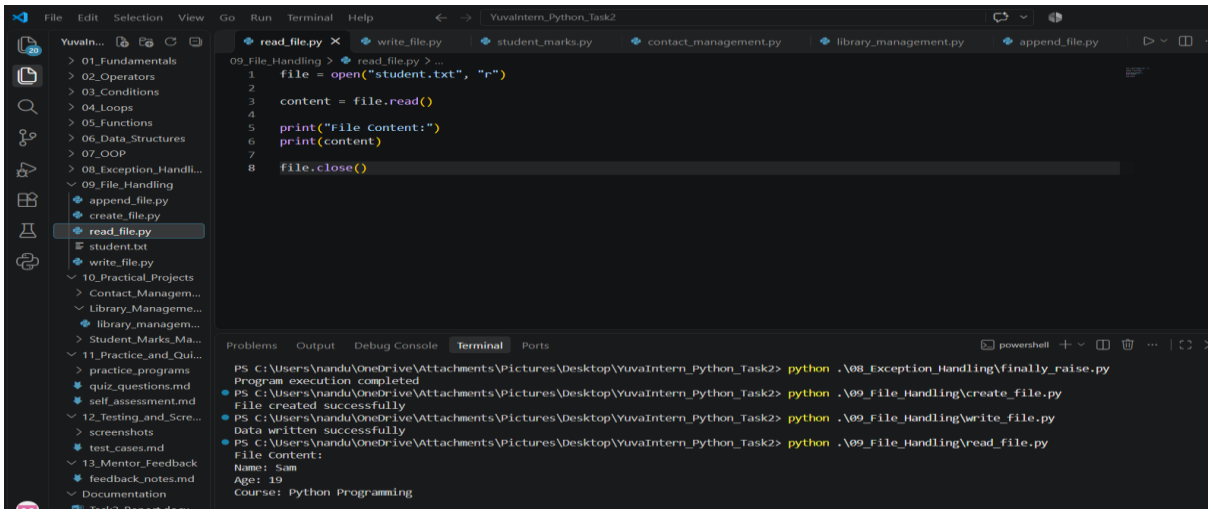


Figure 10: File Handling

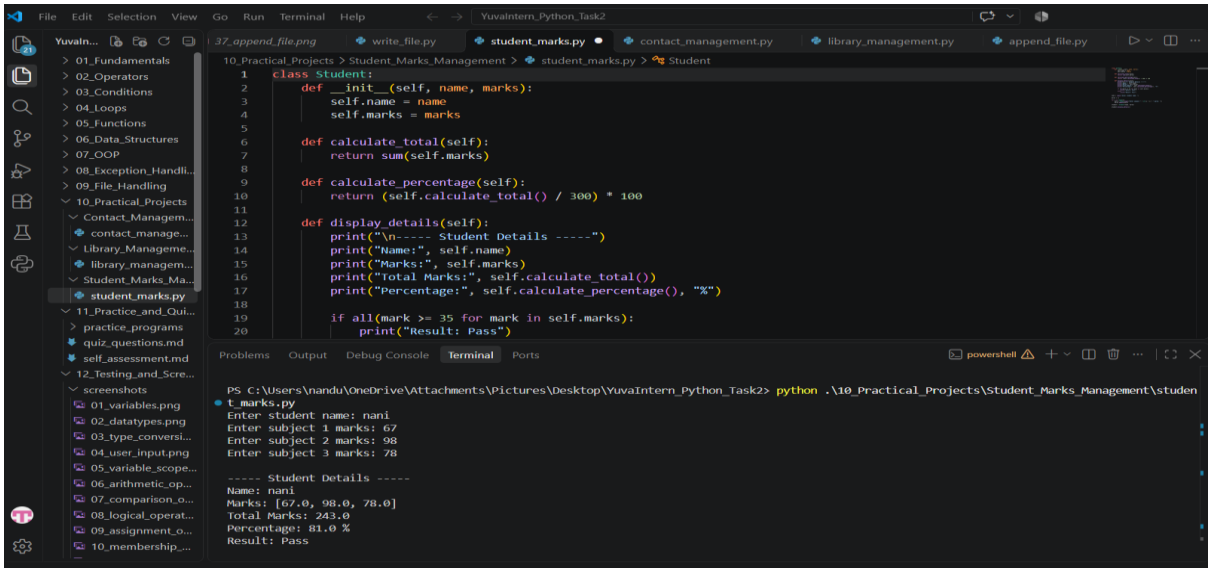


Figure 11: Student Marks Project

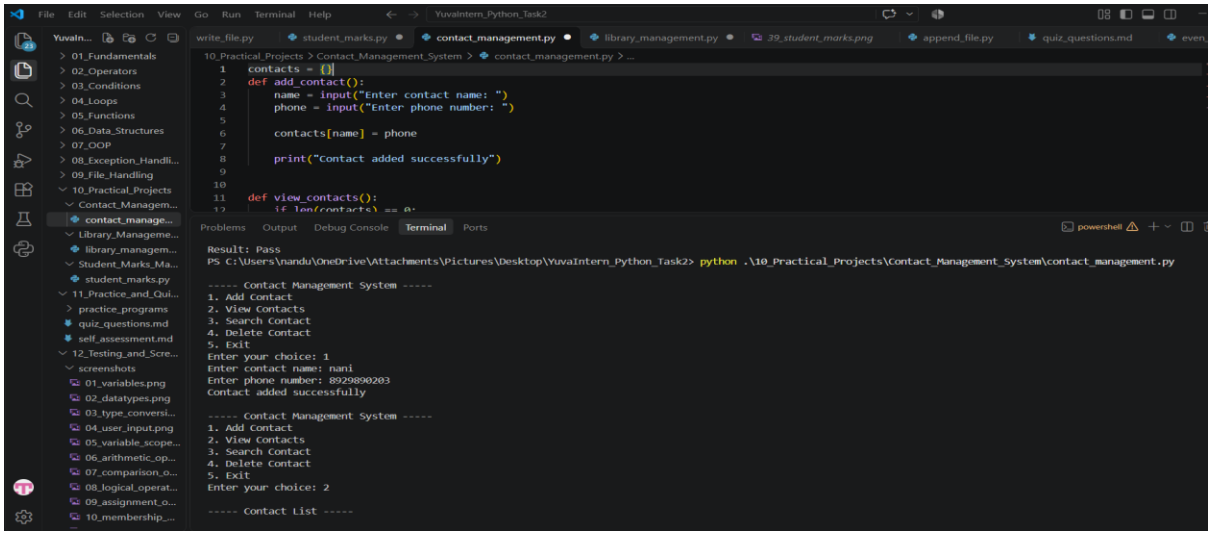


Figure 12: Contact Management Project

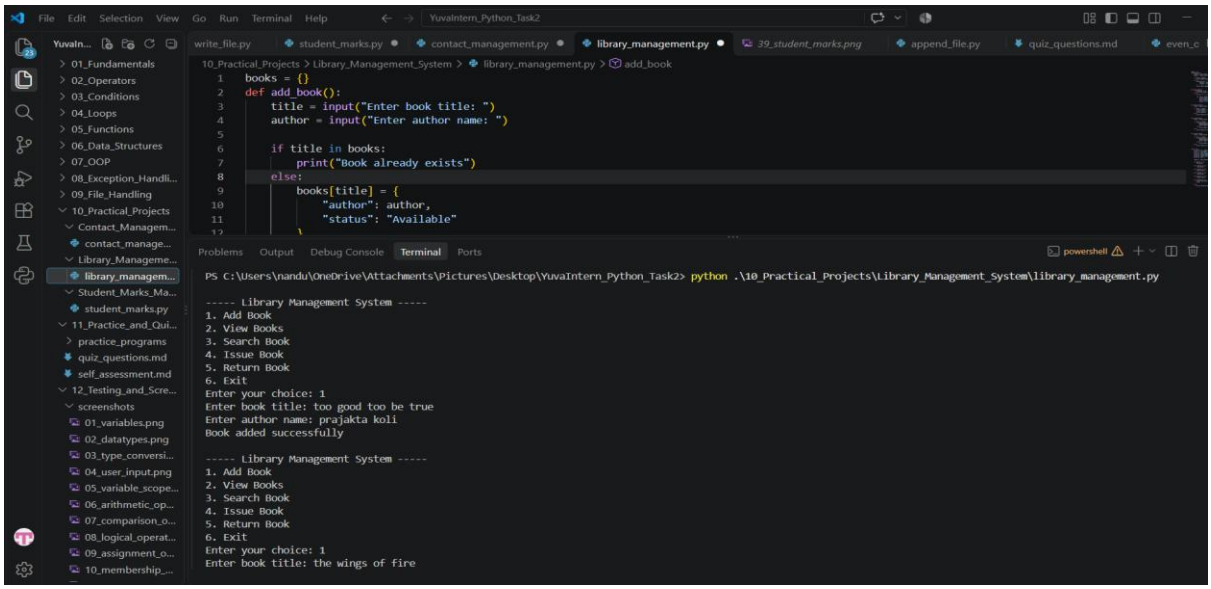


Figure 13: Library Management Project

16. LEARNING OUTCOMES

Through this task, I gained practical experience in:

- Python programming fundamentals.
- Writing and executing Python programs.
- Using loops and functions.
- Working with Python data structures.
- Implementing object-oriented programming.
- Handling runtime exceptions.
- Performing file operations.
- Developing basic real-world applications.
- Debugging and testing programs.

17. VERSION CONTROL USING GIT AND GITHUB

Git and GitHub were used to manage the project files, track changes, and maintain the project repository online. Git helped maintain version history, while GitHub provided an online platform to store and share the project.

Git Commands and Their Explanations

1. **git init**

Initializes a new Git repository in the project folder. It creates the necessary repository information to start tracking changes.

2. **git status**

Displays the current status of the working directory and shows modified, staged, and untracked files.

3. **git add .**

Adds all changes in the current project directory to the staging area.

4. **git add -A**

Stages all changes, including new files, modified files, and deleted files.

5. **git commit -m "message"**

Saves the staged changes as a commit with a meaningful message. This helps maintain a record of project updates.

6. **git branch -M main**

Renames the current branch to main.

7. **git remote add origin repository_url**

Connects the local Git repository to the remote GitHub repository using its URL.

8. **git push -u origin main**

Uploads the committed project files to the main branch of the GitHub repository and establishes the upstream tracking relationship.

9. **git push**

Uploads new local commits to the connected GitHub repository.

Outcome

Using Git and GitHub helped organize the project, maintain version history, track modifications, and publish the completed Python internship project online.

18. CONCLUSION

The YuvaIntern Python Developer Internship – Task 2 provided me with valuable practical experience in Python programming. Throughout this task, I learned and implemented important concepts such as variables, data types, operators, conditional statements, loops, functions, data structures, and object-oriented programming.I also gained practical knowledge of exception handling and file handling by developing and executing different Python programs. Additionally, I developed three mini-projects: Student Marks Management System, Contact Management System, and Library Management System.

Using Visual Studio Code, Git, and GitHub helped me improve my coding, debugging, and project organization skills.

Overall, this internship task strengthened my programming fundamentals, logical thinking, and problem-solving abilities. It has provided me with a strong foundation for learning advanced Python concepts and building real-world software applications.

END OF REPORT
